

LAPORAN PRATIUM STRUKTUR DATA

IMPLEMENTASI SINGLE LINKED LIST DALAM JAVA

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr.WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum "Implementasi Queue dan Prinsip FIFO dalam Java" ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Queue yang bekerja dengan prinsip FIFO (First In, First Out), serta penerapannya melalui implementasi menggunakan library bawaan Java (LinkedList), array statis sirkular (circular array), dan studi kasus nyata seperti simulasi antrian loket dan pembalikan urutan data menggunakan kombinasi Stack-Queue. Saya menyadari bahwa pemahaman tentang struktur data antrian merupakan fondasi penting dalam penjadwalan proses, algoritma pencarian lebar (BFS), manajemen buffer, dan pengembangan sistem yang memprioritaskan urutan kedatangan

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya supaya laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Konsep Single Linked List dalam Java.....	5
2.2 Langkah pengerjaan	6
2.3 Latihan	11
BAB III KESIMPULAN.....	13
3.1 Kesimpulan	13
3.2 Saran	13
DAFTAR PUSTAKA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam Dalam pengembangan perangkat lunak, pemilihan struktur data yang tepat sangat memengaruhi efisiensi dan fleksibilitas program. Struktur data array yang telah dipelajari sebelumnya memiliki kelemahan mendasar: ukurannya statis dan operasi penyisipan/penghapusan di posisi tengah memerlukan pergeseran elemen yang mahal ($O(n)$). Untuk mengatasi keterbatasan ini, digunakan struktur data dinamis yang disebut Linked List.

Single Linked List (SSL) adalah struktur data linier yang terdiri dari kumpulan node, di mana setiap node menyimpan data dan referensi (pointer) ke node berikutnya. Tidak seperti array, elemen-elemen dalam linked list tidak disimpan di lokasi memori yang berurutan, melainkan tersebar secara dinamis dan dihubungkan melalui pointer. Karakteristik ini membuat penyisipan dan penghapusan elemen dapat dilakukan dengan kompleksitas $O(1)$ selama referensi ke node yang dituju diketahui, tanpa perlu pergeseran data.

Pada praktikum ini, mahasiswa diminta untuk membangun kelas node, mengimplementasikan operasi dasar SSL (insert di depan, belakang, posisi tertentu; delete head, delete last, delete pada posisi tertentu; search; traversal), serta menerapkannya pada studi kasus sistem antrian rumah sakit. Praktikum ini menekankan pada pemahaman manipulasi pointer dan penanganan kasus-kasus khusus seperti list kosong dan penghapusan elemen tunggal.

1.2 Tujuan

1. Memahami konsep Single Linked List dan perbedaannya dengan array.
2. Mampu mengimplementasikan kelas node dan operasi penyisipan (insert at front, insert at end, insert at position).
3. Mampu mengimplementasikan operasi penghapusan (delete head, delete last, delete at position).
4. Mampu menerapkan algoritma pencarian dan penelusuran (traversal) pada linked list.
5. Menerapkan SSL untuk menyelesaikan studi kasus sistem antrian rumah sakit yang membutuhkan pengelolaan data dinamis.
- 6.

1.3 Manfaat

1. Mahasiswa memahami keunggulan linked list dalam alokasi memori dinamis dan manipulasi data tanpa pergeseran.
2. Mahasiswa terampil melakukan operasi pointer-based untuk mengubah struktur data secara efisien.
3. Mahasiswa mampu mengidentifikasi dan menangani kondisi batas (empty list, single element) untuk mencegah NullPointerException.
4. Mahasiswa memperoleh pengalaman membangun aplikasi simulasi nyata menggunakan linked list sebagai backend penyimpanan data.

BAB II

PEMBAHASAN

2.1 Konsep Single Linked List dalam Java

Single Linked List (SSL) adalah struktur data linier yang tersusun dari node-node yang saling terhubung satu arah. Setiap node memiliki dua komponen:

- Data: Nilai atau objek yang disimpan.
- Next: Referensi (pointer) ke node berikutnya dalam list. Node terakhir memiliki next = null.

Operasi fundamental pada SSL meliputi:

- Insert at Front: Menyisipkan node baru di awal list. Head diperbarui ke node baru.
- Insert at End: Menyisipkan node baru di akhir list. Dilakukan traversal hingga node terakhir, lalu next-nya diarahkan ke node baru.
- Insert at Position: Menyisipkan node pada indeks tertentu; memerlukan traversal ke node sebelum posisi target.
- Delete Head: Menghapus node pertama. Head dipindahkan ke node berikutnya.
- Delete Last: Menghapus node terakhir. Perlu traversal untuk menemukan node kedua-terakhir.
- Delete at Position: Menghapus node pada posisi tertentu dengan menyesuaikan pointer prev.
- Search: Traversal dengan membandingkan data setiap node hingga ditemukan.
- Traversal/Display: Mengunjungi setiap node dari head hingga null.

Kompleksitas waktu operasi:

- Insert/Delete di awal: $O(1)$
- Insert/Delete di akhir: $O(n)$ (karena traversal)
- Insert/Delete di posisi tertentu: $O(n)$
- Search: $O(n)$

2.2 Langkah pengerjaan

1. NodeSSL_2511532029

Program:

```
1 package Pekan5_2511532029;
2
3 public class NodeSSL_2511532029 {
4     int data_2029;
5     NodeSSL_2511532029 next_2029;
6     public NodeSSL_2511532029(int data_2029) {
7         this.data_2029 = data_2029;
8         this.next_2029 = null;
9     }
10
11 }
```

Gambar 2.1

Penjelasan program:

Program ini mendefinisikan kelas node yang menjadi komponen dasar SSL. Atributnya adalah data_2029 bertipe integer dan next_2029 bertipe referensi ke NodeSSL_2511532029. Konstruktor menginisialisasi data dan mengeset next = null.

2. Program TambahSLL_2511532029 Program:

```

1 package Pktn5_2511532029;
2 public class TambahSLL_2511532029 {
3
4     public static NodeSLL_2511532029 insertAtFront(NodeSLL_2511532029 head, int value) {
5         NodeSLL_2511532029 new_node_2029 = new NodeSLL_2511532029(value);
6         new_node_2029.next_2029 = head;
7         return new_node_2029;
8     }
9
10    public static NodeSLL_2511532029 insertAtEnd(NodeSLL_2511532029 head, int value) {
11        NodeSLL_2511532029 new_node_2029 = new NodeSLL_2511532029(value);
12        if (head == null) {
13            return new_node_2029;
14        }
15        NodeSLL_2511532029 last_2029 = head;
16        while (last_2029.next_2029 != null) {
17            last_2029 = last_2029.next_2029;
18        }
19        last_2029.next_2029 = new_node_2029;
20        return head;
21    }
22
23    static NodeSLL_2511532029 getNode_2029(int data) {
24        return new NodeSLL_2511532029(data);
25    }
26
27    public static NodeSLL_2511532029 insertPos(NodeSLL_2511532029 headNode_2029, int position, int value) {
28        NodeSLL_2511532029 head = headNode_2029;
29        if (position < 1)
30            System.out.println("Invalid position");
31        if (position == 1) {
32            NodeSLL_2511532029 new_node_2029 = new NodeSLL_2511532029(value);
33            new_node_2029.next_2029 = head;
34            return new_node_2029;
35        } else {
36            NodeSLL_2511532029 temp = head;
37            int pos = position;
38            while (pos > 1) {
39                if (temp == null) break;
40                temp = temp.next_2029;
41            }
42            if (temp == null || temp.next_2029 == null && pos > 1) {
43                System.out.println("Posisi di luar jangkauan");
44                return head;
45            }
46            NodeSLL_2511532029 new_node = getNode_2029(value);
47            new_node.next_2029 = temp.next_2029;
48            temp.next_2029 = new_node;
49            return head;
50        }
51    }
52
53    public static void printList(NodeSLL_2511532029 head) {
54        NodeSLL_2511532029 curr_2029 = head;
55        while (curr_2029 != null) {
56            System.out.print(curr_2029.data_2029);
57            if (curr_2029.next_2029 != null)
58                System.out.print(" -> ");
59            curr_2029 = curr_2029.next_2029;
60        }
61        System.out.println();
62    }
63
64    public static void main(String[] args) {
65        NodeSLL_2511532029 head = new NodeSLL_2511532029(2);
66        head.next_2029 = new NodeSLL_2511532029(3);
67        head.next_2029.next_2029 = new NodeSLL_2511532029(5);
68        head.next_2029.next_2029.next_2029 = new NodeSLL_2511532029(6);
69
70        System.out.println("Sebelum berantai awal: ");
71        printList(head);
72
73        System.out.println("Tambah 1 simpul di depan: ");
74        int data = 1;
75        head = insertAtFront(head, data);
76        printList(head);
77
78        System.out.println("Tambah 1 simpul di belakang: ");
79        int data2 = 7;
80        head = insertAtEnd(head, data2);
81        printList(head);
82
83        System.out.println("Tambah 1 simpul ke data 4: ");
84        int data3 = 4;
85        int pos = 4;
86        head = insertPos(head, pos, data3);
87        printList(head);
88    }
89 }

```

Gambar 2.2
Penjelasan program:

Program ini mengimplementasikan tiga jenis operasi penyisipan:

- insertAtFront: Membuat node baru, mengarahkan next-nya ke head lama, lalu mengembalikan node baru sebagai head.
- insertAtEnd: Jika list kosong, mengembalikan node baru sebagai head. Jika tidak, melakukan traversal hingga node terakhir, lalu menyambungkan node baru.
- insertPos: Menyisipkan node pada posisi tertentu. Jika posisi == 1, sama dengan insertAtFront. Jika tidak, traversal hingga node ke-(pos-1), lalu menyisipkan setelahnya. Validasi posisi dilakukan untuk mencegah error.

Metode printList mencetak seluruh data dengan format data1-->data2-->....

Pada main, list awal berisi 2,3,5,6. Kemudian dilakukan:

1. Insert 1 di depan → list menjadi 1,2,3,5,6
2. Insert 7 di belakang → 1,2,3,5,6,7
3. Insert 4 pada posisi 4 → 1,2,3,4,5,6,7

Analisis: Seluruh operasi insert berjalan sesuai ekspektasi. Kompleksitas insert depan $O(1)$, insert belakang $O(n)$, insert pos $O(n)$.

Output:

Serantai berantai awal:
2-->3-->5-->6
tambah 1 simpul di depan:
1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

Gambar 2.3

3. Program PencarianSSL_2511532029

Program:

```
1 package Pekan5_2511532029;
2
3 public class PencarianSSL_2511532029 {
4     static boolean searchKey_2029 (NodeSSL_2511532029 head, int key) {
5         NodeSSL_2511532029 curr_2029 = head;
6         while (curr_2029 != null) {
7             if (curr_2029.data_2029 == key)
8                 return true;
9             curr_2029 = curr_2029.next_2029;
10        }
11        return false;
12    }
13
14    public static void traversal (NodeSSL_2511532029 head) {
15        NodeSSL_2511532029 curr_2029 = head;
16        while (curr_2029 != null) {
17            System.out.print(" " + curr_2029.data_2029);
18            curr_2029 = curr_2029.next_2029;
19            System.out.println();
20        }
21    }
22
23    public static void main(String[] args) {
24        NodeSSL_2511532029 head = new NodeSSL_2511532029(14);
25        head.next_2029 = new NodeSSL_2511532029(21);
26        head.next_2029.next_2029 = new NodeSSL_2511532029(13);
27        head.next_2029.next_2029.next_2029 = new NodeSSL_2511532029(30);
28        head.next_2029.next_2029.next_2029.next_2029 = new NodeSSL_2511532029(10);
29        System.out.print("Penelusuran SSL : ");
30        traversal(head);
31
32        int key = 30;
33        System.out.print("cari data " + key + "=");
34        if (searchKey_2029(head, key))
35            System.out.println("Ketemu");
36        else
37            System.out.println("tidak ada");
38    }
39 }
40
```

Gambar 2.4

Penjelasan program:

Program ini berisi metode:

- searchKey_2029: Traversal membandingkan setiap data node dengan key. Mengembalikan boolean.
- traversal: Mencetak semua data untuk menunjukkan isi list.

Pada main, list diisi 14,21,13,30,10. Pencarian dilakukan untuk key = 30. Karena 30 ada di node ke-4, output "Ketemu".

Analisis: Pencarian linear $O(n)$. Traversal mencetak seluruh elemen.

Output:

```
Penelusuran SSL : 14 21 13 30 10
cari data 30=Ketemu
```

Gambar 2.5

4. Program HapusSSL_2511532029

Program:

```
1 package HapusSSL_2511532029;
2
3 public class HapusSSL_2511532029 {
4     public static NodeSSL_2511532029 deleteHead (NodeSSL_2511532029 head) {
5         if (head == null)
6             return null;
7         head = head.next_2029;
8         return head;
9     }
10    public static NodeSSL_2511532029 removeLastNode(NodeSSL_2511532029 head) {
11        if (head == null) {
12            return null;
13        }
14        if (head.next_2029 == null) {
15            return null;
16        }
17        NodeSSL_2511532029 secondLast_2029 = head;
18        while (secondLast_2029.next_2029 != null) {
19            secondLast_2029 = secondLast_2029.next_2029;
20        }
21        secondLast_2029.next_2029 = null;
22        return head;
23    }
24    public static NodeSSL_2511532029 deleteNode(NodeSSL_2511532029 head, int position) {
25        NodeSSL_2511532029 temp_2029 = head;
26        NodeSSL_2511532029 prev_2029 = null;
27        if (temp_2029 == null)
28            return head;
29        if (position == 1) {
30            head = temp_2029.next_2029;
31            return head;
32        }
33        for (int i = 1; temp_2029 != null && i < position; i++) {
34            prev_2029 = temp_2029;
35            temp_2029 = temp_2029.next_2029;
36        }
37        if (temp_2029 != null) {
38            prev_2029.next_2029 = temp_2029.next_2029;
39        } else {
40            System.out.println("Data tidak ada");
41        }
42        return head;
43    }
44    public static void printList (NodeSSL_2511532029 head) {
45        NodeSSL_2511532029 curr_2029 = head;
46        while (curr_2029.next_2029 != null) {
47            System.out.print(curr_2029.data_2029 + "-->");
48            curr_2029 = curr_2029.next_2029;
49        }
50        if (curr_2029.next_2029 == null) {
51            System.out.print(curr_2029.data_2029);
52        }
53        System.out.println("");
54    }
55    public static void main(String[] args) {
56        NodeSSL_2511532029 head = new NodeSSL_2511532029(1);
57        head.next_2029 = new NodeSSL_2511532029(2);
58        head.next_2029.next_2029 = new NodeSSL_2511532029(3);
59        head.next_2029.next_2029.next_2029 = new NodeSSL_2511532029(4);
60        head.next_2029.next_2029.next_2029.next_2029 = new NodeSSL_2511532029(5);
61        head.next_2029.next_2029.next_2029.next_2029.next_2029 = new NodeSSL_2511532029(6);
62        System.out.print("List awal: ");
63        printList(head);
64        head = deleteHead(head);
65        System.out.print("List setelah dihapus: ");
66        printList(head);
67        head = removeLastNode(head);
68        System.out.print("List setelah simpul terakhir di hapus: ");
69        printList(head);
70        int position = 2;
71        head = deleteNode(head, position);
72        System.out.print("List setelah posisi 2 dihapus: ");
73        printList(head);
74    }
75 }
```

Gambar 2.6

Penjelasan program:

Program ini mengimplementasikan tiga operasi penghapusan:

- deleteHead: Mengembalikan head.next, menghapus head lama.
- removeLastNode: Jika head null atau hanya satu node, kembalikan null. Traversal untuk menemukan node kedua terakhir, lalu set next-nya ke null.
- deleteNode: Menghapus node pada posisi tertentu. Jika posisi 1, panggil deleteHead. Jika tidak, traversal dengan mempertahankan prev, setelah ditemukan, sambungkan prev.next ke temp.next.

Pada main, list awal 1,2,3,4,5,6. Kemudian:

1. deleteHead → 2,3,4,5,6
2. removeLastNode → 2,3,4,5
3. deleteNode posisi 2 → 2,4,5 (menghapus node ke-2 yaitu 3)

Output:

```
List awal: 1-->2-->3-->4-->5-->6
List setelah dihapus: 2-->3-->4-->5-->6
List setelah simpul terakhir di hapus: 2-->3-->4-->5-->6
list setelah posisi 2 dihapus: 2-->3-->4-->5-->6
```

Gambar 2.7

2.3 Latihan

Deskripsi Program:

Program ini mensimulasikan sistem antrian pasien di rumah sakit menggunakan struktur data Single Linked List (SSL). Setiap pasien yang mendaftar akan ditambahkan ke dalam antrian dan dipanggil secara berurutan sesuai prinsip FIFO (First In, First Out). Sistem menyediakan fitur pendaftaran pasien, pemanggilan pasien, penampilan daftar antrian, pencarian pasien berdasarkan nama, dan pengecekan status antrian. Implementasi SSL memungkinkan penambahan dan penghapusan data secara dinamis tanpa batasan kapasitas.

1. Program Pasien_2511532029

Program:

```
1 package Pekan5_2511532029;
2
3 public class Pasien_2511532029 {
4     private String namaPasien_2029;
5     private String penyakit_2029;
6     private int nomorAntrian_2029;
7     public Pasien_2511532029 next_2029;
8
9     public Pasien_2511532029(String namaPasien_2029, String penyakit_2029, int nomorAntrian_2029) {
10         this.namaPasien_2029 = namaPasien_2029;
11         this.penakit_2029 = penyakit_2029;
12         this.nomorAntrian_2029 = nomorAntrian_2029;
13         this.next_2029 = null;
14     }
15
16     public String getNamaPasien_2029() {
17         return namaPasien_2029;
18     }
19
20     public String getPenyakit_2029() {
21         return penyakit_2029;
22     }
23
24     public int getNomorAntrian_2029() {
25         return nomorAntrian_2029;
26     }
27 }
```

Gambar 2.8

a) Deskripsi program Pasien_2511532029 (ADT Data)

- Berisi atribut namaPasien_2029, penyakit_2029, dan nomorAntrian_2029 dengan enkapsulasi private.
- Memiliki atribut public next_2029 bertipe Pasien_2511532029 yang berfungsi sebagai referensi ke node berikutnya dalam Single Linked List.
- Konstruktor menerima nama, penyakit, dan nomor antrian; menginisialisasi next_2029 = null.
- Dilengkapi dengan metode getter untuk mengakses atribut private: getNamaPasien_2029(), getPenyakit_2029(), dan getNomorAntrian_2029().
- Kelas ini dirancang sebagai node penyimpan data pasien yang akan disambungkan dalam linked list.

2. Program RumahSakit_2511532029

Program:

```
NodeSSL_251153...  TambahSSL_2511...  PencarianS...

1 package RumahSakit_2511532029;
2
3 import java.util.Scanner;
4
5 public class RumahSakit_2511532029 {
6     private Pasien_2511532029 head_2029;
7     private Pasien_2511532029 tail_2029;
8     private int counterAntrian_2029;
9     private int jumlahPasien_2029;
10
11     public RumahSakit_2511532029() {
12         this.head_2029 = null;
13         this.tail_2029 = null;
14         this.counterAntrian_2029 = 0;
15         this.jumlahPasien_2029 = 0;
16     }
17
18     public void daftarPasien_2029(String nama_2029, String keluhan_2029) {
19         counterAntrian_2029++;
20         Pasien_2511532029 baru = new Pasien_2511532029(nama_2029, keluhan_2029, counterAntrian_2029);
21         if (head_2029 == null) {
22             head_2029 = baru;
23             tail_2029 = baru;
24         } else {
25             tail_2029.next_2029 = baru;
26             tail_2029 = baru;
27         }
28         jumlahPasien_2029++;
29         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counterAntrian_2029);
30     }
31
32     public void panggilPasien_2029() {
33         if (head_2029 == null) {
34             System.out.println("Antrian kosong, tidak ada pasien yang bisa dipanggil.");
35             return;
36         }
37         Pasien_2511532029 pasienDipanggil_2029 = head_2029;
38         System.out.println("=== Pasien Dipanggil ===");
39         System.out.println("Nomor Antrian : " + pasienDipanggil_2029.getNomorAntrian_2029());
40         System.out.println("Nama Pasien : " + pasienDipanggil_2029.getNamaPasien_2029());
41         System.out.println("Keluhan : " + pasienDipanggil_2029.getPenyakit_2029());
42         System.out.println("=====");
43
44         head_2029 = head_2029.next_2029;
45         if (head_2029 == null) {
46             tail_2029 = null;
47         }
48         jumlahPasien_2029--;
49     }
50
51     public void tampilkanAntrian_2029() {
52         if (head_2029 == null) {
53             System.out.println("Antrian saat ini kosong.");
54             return;
55         }
56         System.out.println("=== DAFTAR ANTRIAN ===");
57         Pasien_2511532029 current_2029 = head_2029;
58         int posisi_2029 = 1;
59         while (current_2029 != null) {
60             System.out.println(posisi_2029 + ". No. Antrian: " + current_2029.getNomorAntrian_2029() +
61                 " | Nama: " + current_2029.getNamaPasien_2029() +
62                 " | Keluhan: " + current_2029.getPenyakit_2029());
63             current_2029 = current_2029.next_2029;
64             posisi_2029++;
65         }
66         System.out.println("=====");
67     }
68
69     public void cariPasien_2029(String nama_2029) {
70         if (head_2029 == null) {
71             System.out.println("Antrian kosong, tidak ada pasien.");
72             return;
73         }
74         Pasien_2511532029 current_2029 = head_2029;
75         boolean ditemukan_2029 = false;
76         while (current_2029 != null) {
77             if (current_2029.getNamaPasien_2029().equalsIgnoreCase(nama_2029)) {
78                 System.out.println("Pasien ditemukan!");
79                 System.out.println("Nomor Antrian : " + current_2029.getNomorAntrian_2029());
80                 System.out.println("Nama Pasien : " + current_2029.getNamaPasien_2029());
81                 System.out.println("Keluhan : " + current_2029.getPenyakit_2029());
82                 ditemukan_2029 = true;
83                 break;
84             }
85             current_2029 = current_2029.next_2029;
86         }
87         if (!ditemukan_2029) {
88             System.out.println("Pasien dengan nama '" + nama_2029 + "' tidak ditemukan dalam antrian.");
89         }
90     }
91
92     public void cekStatusAntrian_2029() {
93         if (head_2029 == null) {
94             System.out.println("Status: Antrian kosong.");
95             return;
96         }
97         System.out.println("=== STATUS ANTRIAN ===");
98         System.out.println("Jumlah pasien dalam antrian: " + jumlahPasien_2029);
99         System.out.println("Pasien terdapat: " + head_2029.getNamaPasien_2029() +
100             (No. Antrian " + head_2029.getNomorAntrian_2029() + "));
101         System.out.println("=====");
102     }
103
104     public void jalankanMenu_2029() {
105         Scanner scanner_2029 = new Scanner(System.in);
106         int pilihan_2029;
107         do {
108             System.out.println("=== Antrian Rumah Sakit NDN: 2511532029 ===");
109             System.out.println("1. Daftarkan Pasien (Insert)");
110             System.out.println("2. Panggil Pasien (Delete Head)");
111             System.out.println("3. Tampilkan Antrian (Display)");
112             System.out.println("4. Cari Pasien (Search)");
113             System.out.println("5. Cek Status Antrian");
114             System.out.println("6. Keluar");
115             System.out.println("Pilihlah: ");
116             pilihan_2029 = scanner_2029.nextInt();
117             scanner_2029.nextLine();
118
119             switch (pilihan_2029) {
120                 case 1:
121                     System.out.print("Masukkan Nama Pasien : ");
122                     String nama_2029 = scanner_2029.nextLine();
123                     System.out.print("Masukkan Keluhan : ");
124                     String keluhan_2029 = scanner_2029.nextLine();
125                     daftarPasien_2029(nama_2029, keluhan_2029);
126                     break;
127                 case 2:
128                     panggilPasien_2029();
129                     break;
130                 case 3:
131                     tampilkanAntrian_2029();
132                     break;
133                 case 4:
134                     System.out.print("Masukkan Nama Pasien yang dicari : ");
135                     String cariNama_2029 = scanner_2029.nextLine();
136                     cariPasien_2029(cariNama_2029);
137                     break;
138                 case 5:
139                     cekStatusAntrian_2029();
140                     break;
141                 case 6:
142                     System.out.println("Terima kasih, program selesai.");
143                     break;
144                 default:
145                     System.out.println("Pilihan tidak valid!");
146             }
147         } while (pilihan_2029 != 6);
148         scanner_2029.close();
149     }
150
151     public static void main(String[] args) {
152         RumahSakit_2511532029 r_2029 = new RumahSakit_2511532029();
153     }
154 }
```

Gambar 2.9

a) Deskripsi program RumahSakit_2511532029

- **Struktur Data Utama:** Single Linked List dengan penunjuk head_2029 (node pertama) dan tail_2029 (node terakhir). Variabel counterAntrian_2029 untuk menghasilkan nomor antrian global dan jumlahPasien_2029 menyimpan total pasien dalam antrian.
- **case 1 (Daftarkan Pasien / Insert Tail):** Membuat objek Pasien_2511532029 baru dan menambahkannya di akhir list menggunakan tail. Jika antrian kosong, head dan tail diarahkan ke node baru. Operasi ini memiliki kompleksitas $O(1)$ karena tidak memerlukan traversal.
- **case 2 (Panggil Pasien / Delete Head):** Mengecek apakah antrian kosong (head == null). Jika tidak, menampilkan data pasien di head, lalu menggeser head ke node berikutnya. Jika setelah penghapusan head menjadi null, tail juga di-nullkan. Operasi $O(1)$.
- **case 3 (Tampilkan Antrian / Display):** Melakukan traversal dari head hingga null sambil mencetak nomor antrian, nama, dan keluhan setiap pasien. Menangani kondisi antrian kosong dengan pesan yang aman. Kompleksitas $O(n)$.
- **case 4 (Cari Pasien / Search):** Menerima input nama, melakukan traversal dan membandingkan setiap nama pasien menggunakan equalsIgnoreCase(). Jika ditemukan, menampilkan detail pasien dan menghentikan pencarian. Jika tidak, menampilkan pesan error. Kompleksitas $O(n)$.
- **case 5 (Cek Status Antrian):** Menampilkan jumlah pasien dalam antrian dan informasi pasien terdepan (head) tanpa menghapusnya.
- **Menu Interaktif:** Menggunakan perulangan do-while dan switch-case untuk menampilkan antarmuka berbasis teks yang terus berjalan hingga pengguna memilih opsi keluar. Input diterima menggunakan Scanner.

Output:

```
== Antrian Rumah Sakit NIM: 2511532029 ==
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : abdul
Masukkan Keluhan : gigi
Pasien berhasil didaftarkan! Nomor Antrian: 1

== Antrian Rumah Sakit NIM: 2511532029 ==
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : karim
Masukkan Keluhan : gusi
Pasien berhasil didaftarkan! Nomor Antrian: 2

== Antrian Rumah Sakit NIM: 2511532029 ==
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

=== DAFTAR ANTRIAN ===
1. No. Antrian: 1 | Nama: abdul | Keluhan: gigi
2. No. Antrian: 2 | Nama: karim | Keluhan: gusi
=====
```

Gambar 2.10

```
-- Antrian Rumah Sakit NIM: 2511532029 --
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
=== STATUS ANTRIAN ===
Jumlah pasien dalam antrian: 2
Pasien terdepan: abdul (No. Antrian 1)
=====

== Antrian Rumah Sakit NIM: 2511532029 ==
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari : karim
Pasien ditemukan!
Nomor Antrian : 2
Nama Pasien : karim
Keluhan : gusi

== Antrian Rumah Sakit NIM: 2511532029 ==
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

=== Pasien Dipanggil ===
Nomor Antrian : 1
Nama Pasien : abdul
Keluhan : gigi
=====

== Antrian Rumah Sakit NIM: 2511532029 ==
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih, program selesai.
```

Gambar 2.11

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan pelaksanaan praktikum implementasi Single Linked List dalam Java, dapat disimpulkan bahwa Single Linked List (SSL) merupakan struktur data dinamis yang sangat efisien untuk operasi penyisipan dan penghapusan di awal list karena hanya membutuhkan waktu $O(1)$. Berbeda dengan array yang memerlukan alokasi memori kontigu dan pergeseran elemen, SSL menyimpan node secara tersebar dan menghubungkannya melalui pointer. Seluruh operasi dasar SSL—penyisipan di depan, belakang, dan posisi tertentu; penghapusan di depan, belakang, dan posisi tertentu; serta pencarian dan penelusuran—telah berhasil diimplementasikan melalui kelas `NodeSSL`, `TambahSLL`, `PencarianSSL`, dan `HapusSSL`. Penggunaan penunjuk tail pada operasi penyisipan di belakang terbukti mampu mengoptimalkan kompleksitas waktu menjadi $O(1)$, sebuah peningkatan signifikan yang menjadikan SSL sangat cocok untuk aplikasi antrian.

Penerapan SSL dalam tugas Sistem Antrian Rumah Sakit menunjukkan bahwa konsep linked list dapat diadaptasi secara nyata untuk menyelesaikan permasalahan manajemen antrian. Program `RumahSakit_2511532029` berhasil mengintegrasikan penyisipan pasien di belakang (menggunakan tail) dan pemanggilan pasien dari depan (penghapusan head) dengan kompleksitas $O(1)$, serta menyediakan fitur pencarian dan penampilan antrian yang stabil. Penanganan kondisi khusus seperti list kosong dan penghapusan elemen tunggal telah diterapkan untuk mencegah `NullPointerException` dan menjaga keutuhan data. Secara keseluruhan, praktikum ini memperkuat pemahaman tentang manipulasi pointer, alokasi dinamis, dan implementasi ADT linear yang menjadi fondasi bagi struktur data lanjutan seperti double linked list, stack, dan queue berbasis linked list.

3.2 Saran

1. Penggunaan visualisasi proses penyambungan dan pemutusan pointer melalui animasi atau debug mode akan sangat membantu mahasiswa memahami alur kerja linked list secara lebih intuitif.
2. Pembagian materi pra-praktikum melalui iLearn tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang sebelum menulis kode di laboratorium.

DAFTAR PUSTAKA

- [1] Oracle. "Single Linked List(Java Platform SE 17)". The Java™ Tutorials. 2024. Available:
<https://forums.oracle.com/ords/apexds/post/single-linked-list-6324>